Two Pointers & Sliding Window

Turn $O(n^2)$ pair scans into $O(n)$ walks

Learning goals

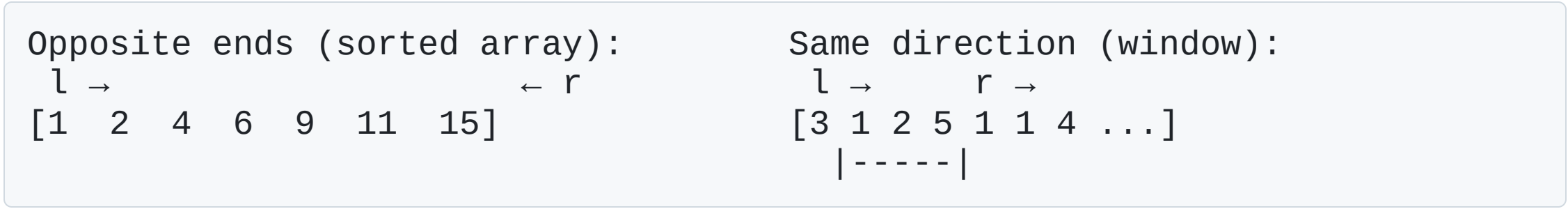
- Recognize when two pointers apply: **monotonicity** of a condition
- Opposite-ends pointers on sorted arrays (pair sums, 3-sum)
- Same-direction pointers: variable-size **sliding windows**
- Count subarrays with a property in O(n)
- Fixed-size windows and merging sorted sequences
- Prove O(n) with an amortized argument

Motivation: CF 279B — Books

n books, reading times $a_i \geq 1$, free time t. Start at any book, read consecutively. Max books?

- Brute force: every start l, extend r → $O(n^2)$, n = 10^5 → 10^10 ×
- Observation: if window [l, r] fits, then [l+1, r] also fits
- So when l moves right, the best r **never moves left**
- Both pointers only move forward → O(n)

Two kinds of two pointers



- Opposite ends: pair problems on **sorted** data
- Same direction: subarrays / windows with a monotone property

Opposite ends: pair with sum x

Sorted a. Find i < j with a_i + a_j = x.

```
int i = 0, j = n - 1;
while (i < j) {
    long long s = a[i] + a[j];
    if (s == x) { /* found */ break; }
    if (s < x) i++;       // need bigger: a[i] is too small for ANY j' <= j
    else       j--;       // need smaller: a[j] is too big for ANY i' >= i
}
```

Each step discards one element forever → ≤ n steps.

Why it's correct

Claim: we never skip a valid pair (i*, j*).

- Suppose i reaches i* first, while j > j*. Then a[i*] + a[j] ≥ a[i*] + a[j*] = x
  - if > x → j-- (move toward j*) ✓; if = x → we found a (possibly different) valid pair ✓
- Symmetric if j reaches j* first
- So the pointers can't jump over the pair

CSES 1640 (Sum of Two Values) needs **original indices** → sort `{value, index}` pairs.

Extension: 3-sum (CSES 1641)

Find three positions with $a_i + a_j + a_k = x$, $n \le 5000$.

- Sort with indices
- Fix i, run two pointers on (i+1 .. n−1) for target x − a_i
- $O(n^2) = 2.5 \cdot 10^7$ ✓ (vs. $O(n^3)$ brute force)

General k-sum: fix k − 2 elements, two pointers for the last two.

Sliding window: the template

```
int l = 0;
for (int r = 0; r < n; r++) {
    add(a[r]);                      // extend window to include r
    while (!valid())                // shrink from the left until valid
        remove(a[l++]);
    // [l, r] is the LONGEST valid window ending at r
    best = max(best, r - l + 1);
}
```

Works when: valid([l, r]) ⇒ valid([l', r]) for all l' ≥ l (shrinking keeps validity).

Why O(n)?

- `r` moves n times
- `l` moves at most n times **in total** (never decreases)
- Each add/remove is O(1) (or O(log n) with a map)

Total O(n) — the inner `while` is amortized, not nested.

Books — code

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long t; cin >> n >> t;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long sum = 0; int l = 0, best = 0;
    for (int r = 0; r < n; r++) {
        sum += a[r];
        while (sum > t) sum -= a[l++];
        best = max(best, r - l + 1);
    }
    cout << best << '\n';
}
```

Trace: Books with a = 3 1 2 1, t = 5

```
r=0 add 3 sum=3              [3]           best=1
r=1 add 1 sum=4              [3 1]         best=2
r=2 add 2 sum=6 >5 drop 3 [1 2] sum=3  best=2
r=3 add 1 sum=4              [1 2 1]       best=3
```

Answer 3.

Counting subarrays (CSES 1660 — Subarray Sums I)

Positive a_i. Count subarrays with sum exactly x.

```
long long sum = 0, cnt = 0; int l = 0;
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum > x) sum -= a[l++];
    if (sum == x) cnt++;
}
```

For each r, at most one l gives sum = x (sums strictly decrease as l grows).

Variant: count windows with sum ≤ x → `cnt += r - l + 1`.

Window with a frequency table: CSES 1141 — Playlist

Longest contiguous segment with **all distinct** songs.

```
map<int,int> cnt;  int l = 0, best = 0;
for (int r = 0; r < n; r++) {
    cnt[k[r]]++;
    while (cnt[k[r]] > 1) cnt[k[l++]]--;
    best = max(best, r - l + 1);
}
```

O(n log n) with `map`; O(n) after coordinate compression.

At most K distinct values

CSES 2428 — Subarray Distinct Values: count subarrays with ≤ k distinct values.

```
map<int,int> cnt; int distinct = 0, l = 0; long long ans = 0;
for (int r = 0; r < n; r++) {
    if (cnt[a[r]]++ == 0) distinct++;
    while (distinct > k)
        if (--cnt[a[l++]] == 0) distinct--;
    ans += r - l + 1;           // all windows ending at r starting in [l, r]
}
```

"Exactly K" = atMost(K) − atMost(K − 1).

Fixed-size windows

Sum (or other aggregate) of every window of length k:

```
long long s = 0;
for (int i = 0; i < n; i++) {
    s += a[i];
    if (i >= k) s -= a[i - k];
    if (i >= k - 1) best = max(best, s);
}
```

Max/min of every window needs a **monotonic deque** (Week 12).

Merging two sorted sequences

```
int i = 0, j = 0;
vector<int> c;
while (i < n || j < m) {
    if (j == m || (i < n && a[i] <= b[j])) c.push_back(a[i++]);
    else                                   c.push_back(b[j++]);
}
```

Same pattern: count pairs a_i < b_j, compare two sorted lists, CSES 1084 Apartments (match applicants to apartments).

When two pointers FAILS

Subarray sum = x with **negative** numbers allowed:

```
a = 2 -1 2,  x = 3     whole array sums to 3
```

Adding an element may decrease the sum → shrinking/expanding is no longer monotone.

Fix: prefix sums + hash map (Week 7, CSES 1661 Subarray Sums II).

Always check the monotonicity condition first.

Two pointers vs. binary search

	Two pointers	Binary search per l
Needs	monotone window property	monotone property + fast range query (prefix sums)
Time	O(n)	O(n log n)
Code	short, careful	simple, robust

Both exploit the same monotonicity — use whichever is easier to get right.

Pitfalls

- Applying it without monotonicity (negative numbers, non-monotone conditions)
- Forgetting to remove a[l] **before** incrementing l (or after — be consistent)
- Off-by-one window length: r – l + 1
- Using `while (l < r)` when the window can become empty (l = r + 1)
- Counting answer `cnt += r - l + 1` overflows `int` — $n^2 / 2 \approx 2 \cdot 10^{10}$

Practice problems

- CSES 1640 Sum of Two Values · 1641 Sum of Three Values · 1660 Subarray Sums I · 1141 Playlist · 2428 Subarray Distinct Values · 1084 Apartments
- Codeforces 279B Books · 6C Alice, Bob and Chocolate · 1354B Ternary String · 701C They Are Everywhere · 660C Hard Process
- AtCoder ABC098 D Xor Sum 2
- Codeforces EDU "Two Pointers Method" course (steps 1–3)

Summary

- Two pointers exploit monotonicity to replace a nested loop with an amortized O(n) walk
- Opposite ends: sorted pair/k-sum problems
- Sliding window: add right, shrink left while invalid; count with r − l + 1
- Check the precondition: shrinking a valid window must keep it valid
- Next week: **prefix sums & difference arrays** — handles the cases two pointers can't